

Hydro Model Architectures

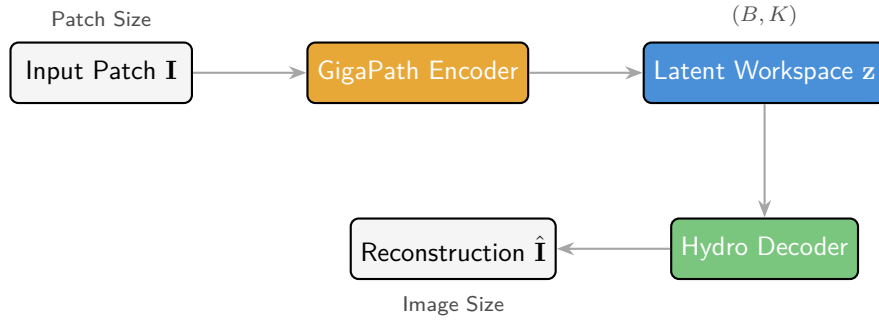
autopath

1 Overview

The **Hydro** datablock provides a flexible framework for latent-to-image decoders. It is designed to invert feature extractors (e.g., GigaPath) by mapping their embeddings back to the original image space.

1.1 Large-scale System Architecture

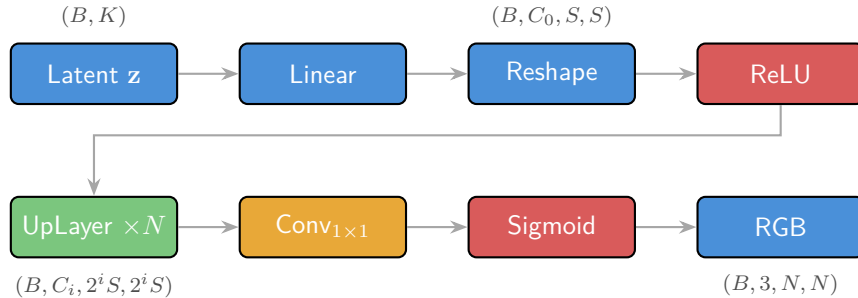
In the context of the overall pipeline, **Hydro** serves as the generative component that transforms compressed semantic embeddings into spatial reconstructions. The global latent vector $\mathbf{z}$ is the primary interface between the large-scale encoder and the decoder.



The latent vector $\mathbf{z}$ is produced by a fixed (or fine-tuned) backbone that summarizes the semantic content of a histology slide or patch. This vector provides the global context used by **Hydro** features like Feature Modulation to inform the progressive upsampling process.

2 Model_CNN: CNN-based Decoder

Model_CNN is the baseline architecture, using progressive upsampling to reconstruct $N \times N$ RGB images from K -dimensional latent vectors.



2.1 Architectural Improvements to Combat Blur

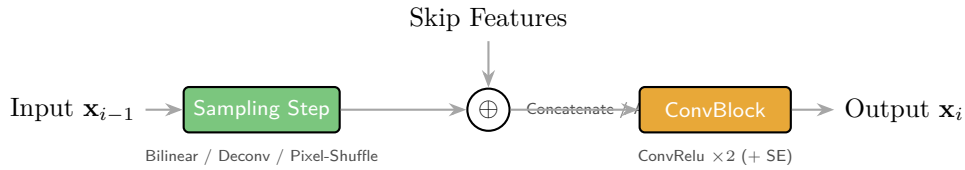
To address the blurriness observed in `Model_CNN` outputs, several optional components can be enabled. These features prioritize the preservation and enhancement of high-frequency information and are situated within the progressive upsampling loop.

2.1.1 Overview of Improved Upsampling Path

where $\mathbf{x}_i \in \mathbb{R}^{C_i \times H_i \times W_i}$.

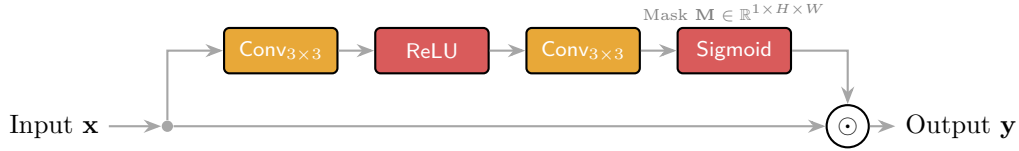
2.1.2 Internal Architecture of UpLayer

The `UpLayer` is the fundamental building block of the CNN decoder. It handles spatial upsampling, optional skip connections from an encoder, and local feature refinement.



2.1.3 Spatial Attention Gates (SAG)

The `SpatialAttentionGate` identifies and emphasizes salient spatial features (e.g., edges and textures). It is situated at the end of each upsampling stage to refine the newly generated spatial details.



The gate receives the upsampled feature map $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$. In its enhanced version, it employs 3×3 convolutions to capture better spatial context when computing the mask, effectively identifying sharp transitions like edges and textures which are then selectively amplified.

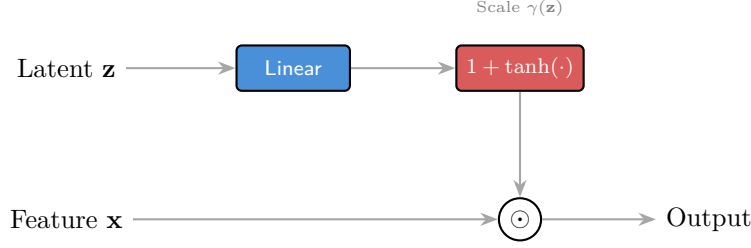
The mask generator sub-network learns a **pixelwise** spatial mask $\mathbf{M}$ where each value $\in [0, 1]$. This single-channel mask is shared across all C channels. The output is computed via the Hadamard product (element-wise multiplication $\odot$) with broadcasting:

$$\mathbf{y}_{c,h,w} = \mathbf{x}_{c,h,w} \times \mathbf{M}_{1,h,w}$$

By focusing representational capacity on these spatially-encoded transitions, SAG effectively reduces smoothing artifacts.

2.1.4 Feature Modulation

Feature modulation allows the global latent vector $\mathbf{z}$ to dynamically scale the intensity of intermediate feature maps. This provides a mechanism for the latent representation to globally influence spatial details at every upsampling stage.



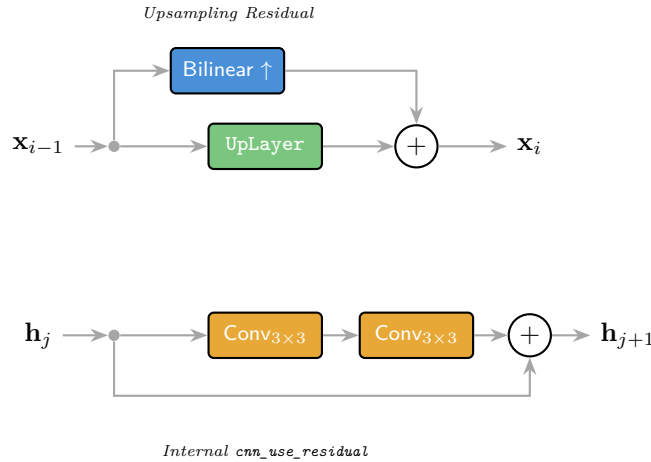
This global conditioning ensures that the reconstructed image maintains sharpening consistency across different scales, as the latent vector can “boost” specific frequency components needed for a sharp reconstruction.

2.1.5 Residual Architectures

Two distinct types of residual connections are available to improve gradient flow and preserve signal fidelity.

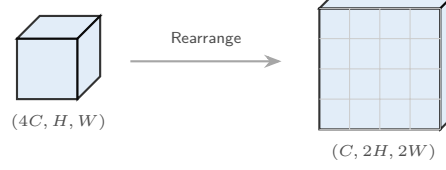
Upsampling Residuals (`cnn_use_residual_upsampling`) This resolution-skipping connection operates **across** an upsampling block. It provides a direct path for low-frequency content by upsampling the previous stage’s feature map (typically via bilinear interpolation) and adding it to the current stage’s output.

Internal Residuals (`cnn_use_residual`) This connection operates **within** a single ConvBlock at a constant resolution. It adds the block’s input directly to its output ($y = f(x) + x$), helping gradients propagate through deep stacks of convolutions without attenuation or blurring.



2.1.6 Pixel-Shuffle Upsampling (`cnn_use_pixel_shuffle`)

Instead of using bilinear interpolation or standard transposed convolutions (deconvolution), the model can be configured to use **Pixel-Shuffle** (Depth-to-Space) upsampling. This technique rearranges channels into higher spatial resolution ($2 \times \uparrow$), which often results in significantly sharper edges as it avoids the interpolation artifacts typical of standard upsampling kernels.

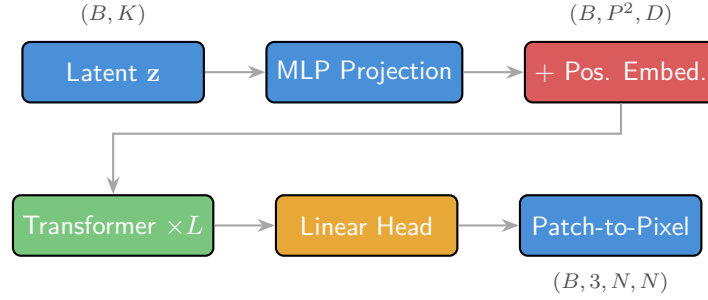


3 Model_ViT: Vision Transformer (ViT) Decoder

Model_ViT adopts a transformer-based architecture to model long-range dependencies and produce sharper reconstructions by operating on a tokenized representation of the image.

3.1 Architecture Overview

The architecture follows a “latent token to patch” approach:



3.1.1 Tokenization and Projection

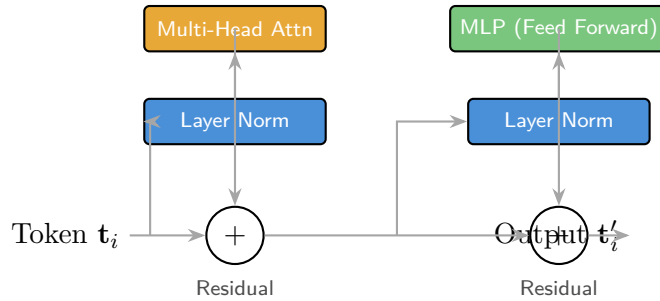
The latent vector $\mathbf{z}$ is projected into a sequence of $M = (N/p)^2$ tokens, where p is the patch size (e.g., 16).

$$\mathbf{z} \xrightarrow{\text{MLP}} \mathbf{T} \in \mathbb{R}^{M \times D}$$

Learnable 2D positional embeddings are added to provide spatial context to the transformer.

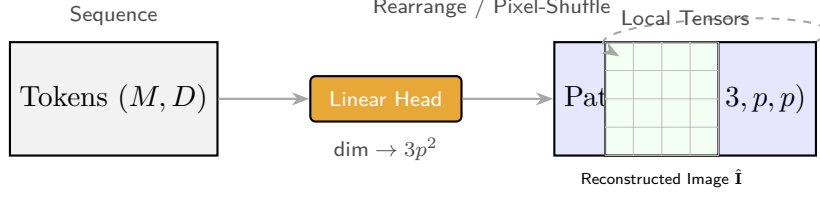
3.1.2 Transformer Backbone

A series of L standard Transformer blocks (Multi-Head Self-Attention + MLP) process the token sequence. This allows information to propagate globally across the image grid.



3.1.3 Reconstruction Head and Patch-to-Pixel

Each token $\mathbf{t}_i \in \mathbb{R}^D$ is projected to a patch $\mathbf{P}_i \in \mathbb{R}^{3 \times p \times p}$ using a linear layer. The patches are then rearranged to form the final image $\mathbf{I} \in \mathbb{R}^{3 \times N \times N}$.



4 System Configuration

The Hydro datablock uses a shared configuration dataclass. Parameters are grouped by their scope.

4.1 General Parameters

Parameter	Default	Description
latent_dim	1536	Input latent vector dimension K
image_size	256	Output resolution $N \times N$
model	'cnn'	Model architecture identifier ('cnn' or 'vit')
loss_type	'mse'	Primary reconstruction loss function (mse, l1, l0, lpips, grad, ssim)
ssim_companion_weight	0.8	Weight α for companion loss when using SSIM
ssim_companion_loss	'lpips'	Companion loss type (l1, mse, lpips)

4.2 Model_CNN Parameters

Parameter	Default	Description
cnn_initial_size	8	Initial spatial resolution $S \times S$
cnn_hidden_channels	256	Base channel count for convolutional layers
cnn_channel_multipliers	(8,4,2,1,1)	Scale factor for channels at each stage
cnn_use_bilinear_upsampling	True	Use bilinear interpolation for upsampling
cnn_use_residual_upsampling	False	Enable resolution-skipping connections
cnn_use_pixel_shuffle	False	Use sub-pixel convolution (Depth-to-Space)
cnn_use_residual	False	Enable internal block residuals ($y = f(x) + x$)
cnn_use_spatial_attention_gates	False	Enable Spatial Attention Gates (SAG)
cnn_use_feature_modulation	False	Enable latent-based scaling

4.3 Model_ViT Parameters

Parameter	Default	Description
vit_patch_size	16	Pixel size of each tokenized patch (p)
vit_hidden_channels	256	Transformer embedding dimension D
vit_n_layers	12	Number of Transformer encoder blocks L
vit_n_heads	8	Number of attention heads
vit_dim_feedforward	1024	Dimension of MLP in Transformer block

5 Sharpness Enhancement Design

Several model-independent mechanisms are provided to improve the visual fidelity of reconstructions, specifically targeting the suppression of blur.

5.1 Perceptual Scaling

The `lpips` loss relies on a pre-trained VGG feature extractor. Since the decoder typically outputs pixel values in the range $[0, 255]$, while VGG expects inputs normalized to $[0, 1]$, an internal scaling factor is applied:

$$\hat{\mathbf{I}}_{\text{perceptual}} = \frac{\hat{\mathbf{I}}}{255.0}$$

This ensuring that the perceptual gradients are correctly scaled relative to the learned weights.

5.2 Gradient Penalization (`grad`)

To explicitly reward the preservation of high-frequency edges, the model can minimize the L_1 distance between the spatial gradients of the predicted and target images:

$$\mathcal{L}_{\text{grad}} = \|\nabla_x \hat{\mathbf{I}} - \nabla_x \mathbf{I}^*\|_1 + \|\nabla_y \hat{\mathbf{I}} - \nabla_y \mathbf{I}^*\|_1$$

where ∇ represents the Sobel operator (or standard finite difference). Unlike MSE, which tends to average out high-frequency noise and produce blurry boundaries, Gradient Loss forces the model to match the local contrast transitions.

5.3 Structural Similarity Index (`ssim`)

The Structural Similarity Index (SSIM) is used as a differentiable loss function to capture perceived changes in structural information, luminance, and contrast. Unlike pixel-wise losses (MSE/L1), SSIM is insensitive to uniform changes in brightness and better matches human visual perception of similarity:

$$\text{SSIM}(\mathbf{x}, \mathbf{y}) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

The model minimizes $1 - \text{SSIM}$, promoting the reconstruction of realistic textures and structural patterns.

The SSIM loss is typically combined with a perceptual or pixel-wise companion loss (e.g., LPIPS or L_1) to ensure intensity consistency and provide more stable gradients:

$$\mathcal{L}_{\text{total}} = (1 - \alpha)\mathcal{L}_{\text{ssim}} + \alpha\mathcal{L}_{\text{comp}}$$

where α is the `ssim_companion_weight` and $\mathcal{L}_{\text{comp}}$ is the value of `ssim_companion_loss` (`l1`, `mse`, or `lpips`).